

DOCKER & DEVOPS

Scenario-Based Practice Questions

8 Practical Scenarios · Complete Solutions · LPU DevOps Lab

Docker Basics

Dockerfile

Docker Compose

Docker Swarm

Docker Networks

CI/CD Jenkins

Storage & Volumes

Microservices

This booklet contains eight scenario-based Docker and DevOps questions modelled on practical lab exercises. Each question presents a real-world situation, lists the tasks you must complete, and is followed by a complete solution with explanation. Topics span the full practical syllabus: installation, images, volumes, Compose, Swarm, networking, CI/CD with Jenkins, and microservices.

Q1

Docker Basics

Container Exec & File Operations

The Missing Report File

SCENARIO

You are a junior DevOps engineer at a logistics company. A containerised Python app running as 'report-app' generates a daily report at /app/reports/daily.txt. Your manager needs that file on the Windows desktop right now, but you have no volume mount configured. You must also push an updated config.json into the container without restarting it, then confirm both files exist inside.

TASKS

1. Enter the running container interactively and list the /app/reports directory.
2. Copy daily.txt from the container to C:\Users\HP\Desktop\.
3. Copy an updated config.json from the desktop into the container at /app/config.json.
4. Verify both files exist inside the container using a single exec command.

SOLUTION

Replace with your actual container ID (use docker ps to find it).

```
# Step 1 – enter container and inspect the reports folder
docker exec -it <container_id> ls /app/reports
```

```
# Step 2 – copy the report OUT to the Windows desktop
docker cp <container_id>:/app/reports/daily.txt C:\Users\HP\Desktop\
```

```
# Step 3 – copy updated config IN to the container
docker cp C:\Users\HP\Desktop\config.json <container_id>:/app/config.json
```

```
# Step 4 – verify both files are present
docker exec -it <container_id> ls /app/reports /app/config.json
```

KEY CONCEPT

docker cp works in both directions without any volume mount. Syntax is always: docker cp SOURCE DESTINATION, where the container path is written as container_id:/path. It works on running AND stopped containers, making it ideal for emergency file recovery or one-off config pushes.

Q2

Docker Basics

Environment Variables & Port Mapping

Production nginx with ENV_MODE

SCENARIO

Your team deploys nginx-based frontends in three environments: dev, staging, and production. The app reads an ENV_MODE environment variable to load the correct config. The ops lead asks you to start a production container, expose it on host port 8080, name it 'web-prod', and confirm the variable is set correctly inside the container.

TASKS

1. Write the docker run command to start nginx with ENV_MODE=production.

2. Map host port 8080 to container port 80.
3. Name the container web-prod and run it in detached mode.
4. Verify the environment variable is set using docker exec.

SOLUTION

```
# Run nginx with environment variable, port mapping and name
docker run -d \
  --name web-prod \
  -e ENV_MODE=production \
  -p 8080:80 \
  nginx:latest
```

```
# Verify the env variable is set inside the container
docker exec web-prod printenv ENV_MODE
# Expected output: production
```

```
# Also check all env vars at once
docker exec web-prod env
```

KEY CONCEPT

The `-e` flag injects environment variables at container start. `-p HOST:CONTAINER` maps ports — here, requests to `localhost:8080` are forwarded to port 80 inside the container where nginx listens. Always use `-d` (detached) for long-running services so the terminal is not blocked.

The Database That Forgets Everything

SCENARIO

A startup's PostgreSQL container crashes every few days. Every restart wipes the entire database because no volume is configured. Customers are losing their order history. You must fix this immediately by attaching a named volume, then demonstrate that data survives a stop-start cycle.

TASKS

1. Run PostgreSQL 15 with a named volume `pg_data` mapped to `/var/lib/postgresql/data`.
2. Insert a test record into the database.
3. Stop and remove the container, then start a new container using the same volume.
4. Verify the test record still exists. List all Docker volumes on the system.

SOLUTION

```
# Run postgres with a named volume
docker run -d \
  --name pg-prod \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=appdb \
  -v pg_data:/var/lib/postgresql/data \
  postgres:15
```

```
# Insert a test record
docker exec -it pg-prod psql -U postgres -d appdb \
  -c "CREATE TABLE orders(id SERIAL, item TEXT);"
docker exec -it pg-prod psql -U postgres -d appdb \
  -c "INSERT INTO orders(item) VALUES ('Widget A');"
```

```
# Stop and REMOVE the container (data would be lost without a volume)
docker stop pg-prod
docker rm pg-prod
```

```
# Start a brand-new container reusing the SAME volume
docker run -d \
  --name pg-prod-new \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=appdb \
  -v pg_data:/var/lib/postgresql/data \
  postgres:15

# Verify data survived
docker exec -it pg-prod-new psql -U postgres -d appdb \
  -c "SELECT * FROM orders;"

# List all volumes
docker volume ls
```

KEY CONCEPT

Named volumes live on the Docker host independently of any container. When you `rm` a container its writable layer is deleted, but a named volume is untouched. Any new container that mounts the same volume sees exactly the same data — this is the correct pattern for stateful services like databases.

Q4

Dockerfile

Image Creation & Layer Optimisation

Containerise the Flask Inventory App

SCENARIO

The development team built a Python Flask inventory service. It has a `requirements.txt` with 12 dependencies. Every time a developer rebuilds the image after a small code change, `pip` re-downloads all 12 packages (taking 3 minutes). You must write an optimised Dockerfile that caches the dependency layer separately from the source code layer.

TASKS

1. Choose `python:3.11-slim` as the base image.
2. Copy and install `requirements.txt` BEFORE copying application code.
3. Expose port 5000 and set the correct CMD to start Flask.
4. Build the image tagged as `inventory-app:1.0` and run it on port 5000.

SOLUTION

```
# Dockerfile (save in your project root)
FROM python:3.11-slim

WORKDIR /app

# Install dependencies FIRST – this layer is cached
# and only rebuilt when requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code AFTER deps – only this layer rebuilds on code edits
COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

```
# Build the image
docker build -t inventory-app:1.0 .

# Run the container
docker run -d \
  --name inventory \
  -p 5000:5000 \
  inventory-app:1.0

# Test it
curl http://localhost:5000/health
```

KEY CONCEPT

Docker builds images layer by layer and caches each one. If a layer's input files have not changed, Docker reuses the cached layer and skips the RUN step. By copying requirements.txt and running pip install before copying the rest of the app, you ensure the slow install step is only triggered when dependencies actually change — not on every code edit. This reduces rebuild time from ~3 min to ~5 sec.

Full-Stack App Crashes on Startup

SCENARIO

You inherit a three-tier application: a React frontend (Node 18), a Java Spring Boot backend (Eclipse Temurin 17), and PostgreSQL 15. When you run `docker compose up`, the backend crashes with 'connection refused' because Spring Boot tries to connect to Postgres before it is ready. You must write a `docker-compose.yml` that enforces the correct startup order using healthchecks.

TASKS

1. Define three services: frontend on port 3000, backend on port 8080, db on port 5432.
2. Add a healthcheck to the db service using `pg_isready`.
3. Make the backend depend on db with condition: `service_healthy`.
4. Pass DB credentials to the backend as environment variables and use a named volume for Postgres data.

SOLUTION

Save this as `docker-compose.yml` in your project root, then run: `docker compose up -d`

```
version: "3.9"
services:
  frontend:
    image: node:18
    container_name: frontend-app
    working_dir: /app
    volumes:
      - ./frontend:/app
    ports:
      - "3000:3000"
    command: sh -c "npm install && npm start"
    depends_on:
      - backend
  backend:
    image: eclipse-temurin:17
    container_name: backend-app
    working_dir: /app
    volumes:
      - ./backend:/app
    ports:
      - "8080:8080"
    command: java -jar app.jar
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: mydb
      DB_USER: postgres
      DB_PASSWORD: password
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:15
    container_name: db
    working_dir: /app
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    healthcheck:
      test: pg_isready -U postgres -d mydb
      interval: 10s
      timeout: 5s
      retries: 10
```

```

db:
  condition: service_healthy
db:
  image: postgres:15
  container_name: postgres-db
  restart: always
  environment:
    POSTGRES_DB: mydb
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    retries: 5
  volumes:
    - db_data:/var/lib/postgresql/data
  ports:
    - "5432:5432"
  volumes:
    db_data:

# Useful Compose commands
docker compose up -d # start all services
docker compose ps # check status
docker compose logs backend # debug backend
docker compose down -v # stop and remove volumes

```

KEY CONCEPT

`depends_on` alone only waits for the container to start — it does NOT wait for the process inside to be ready. `condition: service_healthy` tells Compose to delay the backend until Postgres passes its healthcheck (`pg_isready` exits 0). All three services share a default Compose bridge network and can reach each other by service name — that is why `DB_HOST` is set to `'db'`, not an IP address.

Q6

Docker Compose

WordPress Deployment

Marketing Blog — Up by End of Day

SCENARIO

The marketing director needs a WordPress blog live in under an hour. It must use MySQL 8 as the database, be accessible on host port 8080, and all uploaded images and database records must survive a server reboot. You have been given a clean Ubuntu VM with Docker and Docker Compose installed.

TASKS

1. Write a `docker-compose.yml` with `wordpress` and `db` (MySQL 8) services.
2. Map WordPress to host port 8080 and pass all required environment variables.
3. Create named volumes for both MySQL data and WordPress uploads.
4. Start the stack and confirm WordPress is accessible via `curl` on port 8080.

SOLUTION

```
version: "3.9"
services:
  wordpress:
    image: wordpress:latest
    container name: wp-blog
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_NAME: wpdb
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
    volumes:
      - wp_data:/var/www/html
    depends on:
      - db
    restart: always
  db:
    image: mysql:8
    container name: wp-mysql
    restart: always
    environment:
      MYSQL_DATABASE: wpdb
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - db_data:/var/lib/mysql
  wp_data:
  db_data:
```

```
# Start the stack
docker compose up -d

# Watch logs until WordPress is ready
docker compose logs -f wordpress

# Confirm WordPress responds
curl -I http://localhost:8080

# Expected: HTTP/1.1 302 Found (redirect to setup wizard)
```

KEY CONCEPT

Both services share the default Compose bridge network, so WordPress reaches MySQL using the service name 'db' as the hostname. The wp_data volume persists uploaded media and plugin files; db_data persists the MySQL database. Without these volumes, every docker compose down would erase all blog content.

API Service Cannot Handle Morning Traffic

SCENARIO

Your company's REST API gets 10x normal traffic every weekday morning. The single container running the API becomes the bottleneck and requests time out. You must initialise a Docker Swarm, deploy the API as a replicated service, scale it up before peak hours, and verify it restarts automatically if a replica crashes.

TASKS

1. Initialise Docker Swarm on the manager node.
2. Deploy api-service with 3 replicas, published on port 5000, with restart on failure.
3. Scale the service to 6 replicas before peak hours.
4. Simulate a crash by killing one task and confirm Swarm restarts it automatically.

SOLUTION

```
# Step 1 – initialise Swarm (run on the manager node)
docker swarm init
# Note the join token printed – use it to add worker nodes
```

```
# Step 2 – create the service with 3 replicas
docker service create \
  --name api-service \
  --replicas 3 \
  --publish published=5000,target=5000 \
  --restart-condition on-failure \
  --restart-max-attempts 5 \
  myapp/api:latest
```

```
# Step 3 – check all replicas are running
docker service ps api-service
```

```
# Step 4 – scale to 6 replicas before peak (zero downtime)
docker service scale api-service=6
# Verify
docker service ps api-service
```

```
# Step 5 – simulate a crash (get a task container ID from 'docker ps')
docker stop <one replica container id>
# Swarm detects the stopped task and starts a replacement
# Watch it recover:
docker service ps api-service
# You will see the stopped task listed as 'Shutdown' and a new one as 'Running'
```

KEY CONCEPT

Docker Swarm's scheduler continuously watches the desired replica count. If a task (container) exits with a failure, `--restart-condition on-failure` tells the scheduler to launch a replacement on any available node. The ingress load balancer automatically distributes traffic across all replicas on port 5000 — no extra configuration needed.

Stop Manually Pushing to Docker Hub

SCENARIO

Your team pushes code to GitHub many times a day. A developer manually runs 'docker build' and 'docker push' each time — taking 10 minutes and often forgotten. You must create a Jenkins pipeline that automatically builds the Docker image, tags it with the Jenkins build number, and pushes it to Docker Hub on every push to main. Docker Hub credentials must be stored securely, never in the Jenkinsfile.

TASKS

1. Store Docker Hub credentials in Jenkins Credential Manager as 'dockerhub-creds'.
2. Write a Jenkinsfile with Checkout, Build, and Push stages.
3. Tag the image with the Jenkins BUILD_NUMBER environment variable.
4. Ensure the pipeline logs out of Docker Hub after every run (success or failure).

SOLUTION

Save this Jenkinsfile in the root of your GitHub repository and create a Jenkins Pipeline job pointing to it.

```
// Jenkinsfile
pipeline {
    agent any
    environment {
        IMAGE = "yourdockerhubuser/mvapp"
        TAG = "${env.BUILD_NUMBER}"
    }
    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build image') {
            steps {
                sh "docker build -t ${IMAGE}:${TAG} ."
                sh "docker tag ${IMAGE}:${TAG} ${IMAGE}:latest"
            }
        }
        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DH_USER',
                    passwordVariable: 'DH_PASS'
                )]) {
                    sh 'echo $DH_PASS | docker login -u $DH_USER --password-stdin'
```

```

sh "docker push ${IMAGE}:${TAG}"
sh "docker push ${IMAGE}:latest"
}
}
}
}

post {
  always {
    sh 'docker logout'
  }
  success {
    echo "Image ${IMAGE}:${TAG} pushed successfully."
  }
  failure {
    echo "Pipeline failed – check build logs."
  }
}
}

# Jenkins setup steps:
# 1. Dashboard > Manage Jenkins > Credentials > Add
# Kind: Username with password
# ID: dockerhub-creds
# Username: <your Docker Hub username>
# Password: <your Docker Hub access token>
#
# 2. New Item > Pipeline > Definition: Pipeline script from SCM
# SCM: Git | Repository URL: <your GitHub repo>
# Branch: */main | Script Path: Jenkinsfile

```

KEY CONCEPT

withCredentials injects the username and password into environment variables that are automatically masked in Jenkins build logs — they never appear as plain text. Tagging with BUILD_NUMBER gives every image a unique, traceable tag so you can roll back to any previous build. The post { always } block ensures docker logout runs even if the push stage fails, preventing credential leakage on the agent.